

CSS Bootcamp, Day 4

Text & Dimensionality

23 September 2026

The Challenge of High-Dimensional Data: In computational social science, text is a primary proxy for culture, ideology, and discourse. But text is unstructured and mathematically intractable in its raw form.

For the Next 2 Hours:

1. **Text Preprocessing:** Transforming raw strings into clean tokens.
2. **Matrix Representation:** Building document-term matrices.
3. **Statistical Weighting:** Accounting for term frequency and specificity (TF-IDF).
4. **Dimensionality Reduction:** SVD mechanics and the link to PCA.
5. **Applied Analysis:** Extracting and visualizing themes from 'dtm_filtered'.

The Text-Processing Pipeline

Before a computer can analyze text, we must systematically strip away noise while preserving semantic structure.

- ▶ **Tokenization:** Splitting a continuous text string into individual units (words or n-grams) using some sort of boundary disambiguation.
- ▶ **Normalization:** Converting text to lowercase, removing punctuation, and stripping numbers.
- ▶ **Stop-Word Removal:** Filtering out high-frequency, low-information words (e.g., *the*, *and*, *of*, *in*).
- ▶ **Stemming / Lemmatization:** Reducing words to their root forms (e.g., "voting", "voted" → "vote").

The Document-Term Matrix (DTM)

Computers understand numbers, not paragraphs. We bridge this gap using the Document-Term Matrix.

- ▶ **Structure:** Rows represent documents (N), and columns represent unique terms/vocabulary (V).
- ▶ **Cell Values:** Counts of how many times term j appears in document i .

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1V} \\ x_{21} & x_{22} & \dots & x_{2V} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \dots & x_{NV} \end{bmatrix}$$

Note: DTMs are typically massive and highly sparse (mostly zeros).

What We've Got vs. What We Want

[1] The pictures from Charlottesville, Va., reveal what pictures of oppression and violence generally do not: the ordinary people who typically perpetuate white supremacy.

[2] Fights broke out Friday night after several hundred torch-bearing men and women rallied on the university's main quadrangle. It was the first in what was expected to be a weekend of such rallies.

[3] Protests of a plan to remove a statue of a Confederate general led to clashes that left at least one person dead and 34 others injured, officials said.

[4] After a white nationalist rally led to melees and the death of a 32-year-old woman, the city tried to recover – as the police, in particular, came in for criticism.

[5] A statue of Robert E. Lee was at the center of divisions in the city well before violence broke out this weekend.

31 Levels: A Ku Klux Klan member was charged with shooting in the direction of a black counterprotester. Another man was charged with beating a 20-year-old black man. ...



Terms		charlottesvill	citi	event	extremist	nationalist	plan	ralli	supremacist	white
Docs	charg									
11	0		0	0	1		0	1	0	1
13	0		0	0	0		1	0	1	0
14	0		0	0	0		0	1	0	0
16	0		1	0	1		1	0	0	1
19	0		1	2	0		0	0	0	1
2	0		0	0	0		0	0	2	0
20	0		0	0	0		0	1	1	0
3	0		0	0	0		0	1	0	0
4	0		0	1	0		1	0	1	0
8	0		1	0	0		0	0	1	1

Figure: From Messy Documents to a Tidy Matrix

A Sample Preprocessing Pipeline (Code)

Let's construct a simple corpus to walk through one possible preprocessing pipeline.

```
texts <- c(doc1 = "We're keeping 5 of the the computers?",
           doc2 = "Wow! He's got 3 laptops...",
           doc3 = "Computers and laptops are both computers.")

library(textclean) #install first
texts <- replace_contraction(texts)

library(quanteda) #install first
toks <- tokens(texts, remove_numbers = TRUE, remove_punct =
  TRUE) |>
  tokens_tolower()
```

A Sample Preprocessing Pipeline (DTM)

Next, we stem the words (Porter algorithm), build our Document-Term Matrix, and drop sparse terms.

```
toks_stem <- tokens_wordstem(toks, language = "en")

dtm <- dfm(toks_stem)

dtm_filtered <- dfm_trim(dtm, min_termfreq = 2)

as.matrix(dtm_filtered)
#      features
#docs  are the comput laptop
# doc1   1   2       1       0
# doc2   0   0       0       1
# doc3   1   0       2       1
```

There is no single "correct" pipeline.

The choice of specific pipeline depends heavily on how you want to *"map"* the text—i.e., what information you need to represent or investigate based on your data quality and research question.

A Note on Stemming:

- ▶ Stemming (like the Porter algorithm) is a blunt instrument. It simply chops off word endings.
- ▶ There are better ways of word normalization, namely **POS-tagging followed by lemmatization** (e.g., using the `udpipe` package in R).
- ▶ Lemmatization uses dictionary definitions and context to return the proper base word.

Term Frequency-Inverse Document Frequency (TF-IDF)

Raw word counts can be misleading: words like "the" appear everywhere, while rare words might be typos. TF-IDF balances this out.

- ▶ **Term Frequency (TF):** How often does a word appear in a specific document?
- ▶ **Inverse Document Frequency (IDF):** Downweights words that appear across many documents in the broader corpus.

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log \left(\frac{N}{\text{df}_t} \right)$$

Result: Highlights words that are locally frequent in a document but globally rare across the corpus.

TF-IDF in Action: The Raw Counts

What do TF-IDF scores actually look like? Let's use our fake DTM. First, the raw counts:

```
as.matrix(dtm_filtered)
#           features
#docs    are the comput laptop
# doc1    1  2      1      0
# doc2    0  0      0      1
# doc3    1  0      2      1
```

Notice that "are" appears in two documents, while "the" appears only in one. "comput" appears three times total, but twice in doc3.

TF-IDF in Action: The Weighted Scores

Now we apply TF-IDF. Watch how the values adjust to reflect *relative* importance.

```
dtm_tfidf <- dfm_tfidf(dtm_filtered)

as.matrix(dtm_tfidf)
#           features
#docs      are      the      comput      laptop
# doc1 0.1760913 0.9542425 0.1760913 0.0000000
# doc2 0.0000000 0.0000000 0.0000000 0.1760913
# doc3 0.1760913 0.0000000 0.3521825 0.1760913
```

- ▶ "the" gets a **high score** in doc1 because it is unique to that document (high IDF).
- ▶ "are" gets a **lower score** where it appears because it is common across multiple documents (low IDF). Same with "laptop."

The Curse of Dimensionality

A standard text corpus might contain $N = 10,000$ documents and $V = 50,000$ unique vocabulary terms.

- ▶ This means every document lives in a **50,000-dimensional vector space**.
- ▶ Most dimensions are zero (sparsity), making distance metrics and regressions computationally unstable.
- ▶ **The Solution:** We need to compress the semantic space while retaining maximum information via matrix factorization.

Singular Value Decomposition (SVD)

SVD is the foundational linear algebra technique behind Latent Semantic Analysis (LSA) and dimension reduction.

Any real $N \times V$ matrix $\mathbf{X}$ can be factored into three matrices:

$$\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$$

- ▶ $\mathbf{U}$ ($N \times r$): Document vectors in the new latent semantic space.
- ▶ $\mathbf{\Sigma}$ ($r \times r$): Diagonal matrix of singular values (weights of importance).
- ▶ $\mathbf{V}^T$ ($r \times V$): Term vectors in the latent semantic space.

By keeping only the top k dimensions, we remove noise and capture underlying discursive themes.

SVD: Low-Rank Approximation & Noise Reduction

Why truncate SVD to k dimensions? The best rank- k approximation of $\mathbf{X}$ is:

$$\mathbf{X}_k = \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^T = \sum_{i=1}^k (\mathbf{u}_i \sigma_i) \mathbf{v}_i^T$$

- ▶ The σ_i singular value captures the variance/importance of the i -th latent dimension.
- ▶ Small singular values largely represent idiosyncratic word choices, typos, and matrix sparsity (noise).
- ▶ Truncating pushes noise to zero while preserving major semantic axes.

How does SVD relate to Principal Component Analysis (PCA)?

- ▶ If we apply SVD directly to the mean-centered (and optionally scaled) DTM $\mathbf{X}^*$:

$$\mathbf{X}^* = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$$

- ▶ The right singular vectors ($\mathbf{V}$) are precisely the **principal component loadings** (variable weights).
- ▶ The scaled left singular vectors ($\mathbf{U}\mathbf{\Sigma}$) are the **principal component scores** (document coordinates).
- ▶ When applied to a DTM, PCA/SVD is known as **latent semantic analysis (LSA)**.

Loading a Corpus with text2map.corpora

Let's examine real text data using the abstracts from the 2020 *Annual Review of Sociology* articles.

Open your R session in Positron and run:

```
install.packages("pacman")
remotes::install_gitlab("culturalcartography/text2map.corpora")

pacman::p_load(tidyverse, text2map, text2map.corpora, quanteda,
  install = T)

# Load the bundled sociological abstracts corpus
df <- load_corpus("corpus_annual_review") |>
  distinct(title, .keep_all = T)
```

Preprocessing

Let's clean our text and build a document-term matrix (DTM) with unigrams and bigrams.

```
# Minimal text preprocessing
corp <- corpus(df, text_field = "abstract", docid_field = "
  title")
toks <- tokens(corp, remove_punct = TRUE) |>
  tokens_tolower() |>
  tokens_ngrams(n = 1:2)

dfm_obj <- dfm(toks)
dfm_obj <- dfm_remove(dfm_obj, pattern = stopwords("en"))
dfm_obj <- dfm_select(dfm_obj, min_nchar = 3)
dfm_obj <- dfm_trim(dfm_obj, min_termfreq = 2)
```

A tiny bit more preprocessing.

```
stops <- stopwords("en")
dfm_obj <- dfm_remove(dfm_obj,
                      pattern = c(paste0("^", stops, "_"),
                                   paste0("_", stops, "$")),
                      valuetype = "regex")
```

Applying PCA to the TF-IDF Matrix in R

We can now weight the DTM by TF-IDF and pass the matrix to PCA to extract orthogonal latent dimensions:

```
# Apply TF-IDF weighting to downweight filler words
tfidf_dfm <- dfm_tfidf(dfm_obj)

# Convert weighted dfm to a standard matrix
mat <- as.matrix(tfidf_dfm)
mat <- mat[, apply(mat, 2, var) > 0]

# Run PCA (centering and scaling TF-IDF scores)
pca_themes <- prcomp(mat, center = T, scale. = T)
```

Deciding on k

How much variance in the original TF-IDF DTM does each latent dimension explain?

```
# Inspect variance explained by each principal component
plot(pca_themes, type = "l", main = "Scree Plot of Latent
     Dimensions")

# Looks at eigenvalues
eigenvalues <- pca_themes$sdev^2
variance_explained <- eigenvalues / sum(eigenvalues) * 100
variance_explained |> round(2)
```

Interpreting Themes

To extract semantic themes, we inspect variable loadings (**V**) and project documents into the new space:

```
# Inspect term loadings for different components
pca_themes$rotation[, 1] %>% #switch out number for different
  dimensions
sort(decreasing = T) %>%
head(10)
```

Visualizing Themes

```
# Install and load an extra package
pacman::p_load(ggrepel, install = T)

# Extract document scores and word loadings
doc_df <- as.data.frame(pca_themes$x[, c(1, 8)])
colnames(doc_df) <- c("PC1", "PC8")
doc_df$title <- rownames(doc_df)

word_df <- as.data.frame(pca_themes$rotation[, c(1, 8)])
colnames(word_df) <- c("PC1", "PC8")
word_df$word <- rownames(word_df)
```

Visualizing Themes

```
# Filter and categorize words by where they load high
threshold <- 0.08
filtered_words <- word_df %>%
  filter(abs(PC1) > threshold | abs(PC8) > threshold) %>%
  mutate(
    dim_type = case_when(
      abs(PC1) > threshold & abs(PC8) > threshold ~ "Both Dims",
      abs(PC1) > threshold ~ "PC1 Only",
      abs(PC8) > threshold ~ "PC8 Only"
    )
  )
```

Visualizing Themes

```
# Calculate scaling factor to align word positions with
  document space
scale_factor <- max(abs(doc_df$PC1)) / max(abs(filtered_words$
  PC1)) * 0.65

filtered_words <- filtered_words %>%
  mutate(PC1_scaled = PC1 * scale_factor,
         PC8_scaled = PC8 * scale_factor)
```

Visualizing Themes

```
# Plot it
plot <- ggplot() +
  geom_text(data = doc_df,
            aes(x = PC1, y = PC8, label = title),
            color = "black", alpha = 0.5, size = 2.5) +
  geom_text_repel(data = filtered_words,
                  aes(x = PC1_scaled, y = PC8_scaled, label =
                      word, color = dim_type),
                  fontface = "bold", size = 3.5,
                  box.padding = 0.35, point.padding = 0.2,
                  max.overlaps = Inf)
```

Visualizing Themes

```
plot + scale_color_manual(values = c(
  "PC1 Only" = "#440154ff",
  "PC8 Only" = "#2a788eff",
  "Both Dims" = "#7ad151ff"
)) +
labs(x = "Principal Component 1",
     y = "Principal Component 8",
     color = "High Loading On:") +
theme_bw() +
theme(
  panel.grid.minor = element_blank(),
  panel.grid.major = element_line(color = "gray90"),
  legend.position = "bottom"
)
```

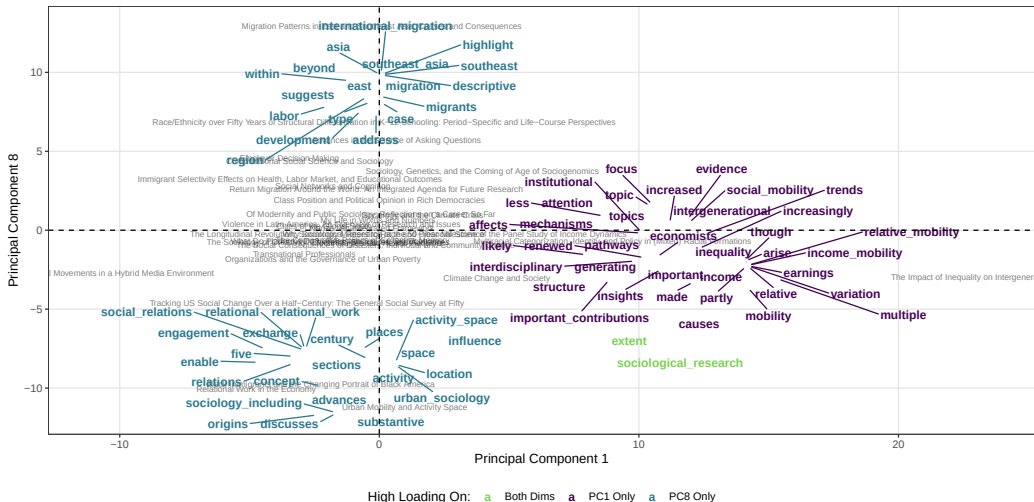


Figure: Biplot with Dimensions 1 and 8



BREAK FOR LUNCH



ENSAE-ENSAI
Groupe

(GENES)



Center for Research in Economics and Statistics
CREST UMR 9194 - 5 avenue Henry Le Chatelier
91120 Palaiseau

Group Application: Latent Semantic Analysis

Follow the instructions here: https://bit.ly/css_bootcamp_day4.